

University of Virginia Course Enrollment as a Matching Game

PIPPA HARRIS AND LOUISA RAWLES

In matching theory, many-to-one matching is a type of matching in which multiple agents on one side match to a single agent on the other side, with each side having preferences over their matchings. Various mechanism designs can be utilized to perform the matching, including Random Serial Dictatorship (RSD) and Deferred Acceptance (DA), which optimize the matching according to criteria like stability and efficiency. In this paper, we consider the matching of many students to a course. Both students and courses have preferences over the other, which we model using data from course rating platforms and vectorized preference orderings. We show that DA using our preference ordering outperforms RSD, highlighting an opportunity for the University of Virginia (UVA) to improve satisfaction in course enrollment mechanisms.

CONTENTS

Abstract	0
Contents	0
1 Introduction	1
2 Data Sourcing	1
2.1 Course Data	2
2.2 Student Data	3
3 Derivation of Preference Orderings	3
3.1 Student Preferences over Courses	3
3.2 Course Preferences over Students	4
4 Simulator	4
4.1 Random Serial Dictator	4
4.2 Deferred Acceptance	4
5 Performance Evaluation	5
6 Conclusions	6
7 Future Work	6
References	6
A Code Base Information	7

1 Introduction

This paper utilizes concepts from many-to-one matching theory, including deferred acceptance and random serial dictator.

The motivation of this paper is the difficulty in the current UVA course enrollment process. Students are separated into groups based on class year and scholar status, then assigned random ordering within these groups. Once this ordering and the corresponding time slots assigned, students may choose courses ahead of time to prepare for their enrollment slot. During their timeslot, students may choose to enroll in any course that still has capacity.

However, this system maintains many painpoints:

- (1) **Difficulty deciding which courses to take:** There are hundreds of course on offer each semester, with scattered information and reviews.
- (2) **No guarantee of course availability:** When your enrollment timeslot comes, there is no guarantee you can get courses that are beneficial to your major or plan of study. Courses that may be required may be full, and you can end up on lengthy waitlists.
- (3) **Mismatched preferences:** In the current system, there is no accounting for student needs or student/course compatibility. The only thing that determines what courses you can take is which courses have space during your enrollment time.
- (4) **No suggesting:** There could be courses that would be a perfect fit for a student's preferences or learning style, but they might have no idea that they exist or have empty seats when their enrollment time comes around.

We aim to address these issues in this paper by integrating what we have learned this semester about matching mechanisms, especially concepts relating to preference ordering and different matching mechanisms and their strengths and weaknesses, which we explore in Section 3 and Section 4.

Another key, UVA-specific concept to introduce is The Course Forum, [cou, 2026], where we source much of our course information. This is a student-run website that aggregates feedback and information about different courses, professors, and sections, and we use this data to develop a model for courses based on feedback from real UVA students.

2 Data Sourcing

To construct course preferences, we propose a crowd-sourcing method to obtain aggregated course data based on reviews and ratings. We chose to use *theCourseForum* for data collection, given its popularity among students for course reviews and peer-inputted information. Additionally, the platform contains data schemas that effectively map to student preferences.

The architecture of our data collection component is composed of layers for extraction, parsing, and finally, structuring. Intermediate JSON files facilitate the flow of data through the pipeline.

The extraction layer is composed of a web scraper implemented using BeautifulSoup4 and Python Requests. The soup object is acquired using the built in HTML parser feature. The reference addresses, href, for each course are extracted from the soup object and cleaned, adding the course prefix and returning the data as a list. We iterate through the list to access and organize course page data into the url, text lines, and the original soup object. The course addresses are generated given the department number as appearing on *theCourseForum*. We note that for application beyond our simulation, an enum object mapping the department mnemonic to *theCourseForum* number would enhance usability. We store the structured url data into `course_links.json`, and immediately use it

Table 1. Course Model

Course ID	Department mnemonic, 4 numbers (XX 1234)
Quota	Integer Course Capacity
Average GPA	4.0 Scale
Department	Engineering Department mnemonic
Reading Score	0-5
Writing Score	0-5
Groupwork Score	0-5
Time Commitment	0-5 hours outside of class

to create course-section dictionary objects that we save into `course_sections.json`, composing the necessary data for parsing.

The parsing layer takes the course-section JSON object as input and contains methods to define course characteristics through its metadata (i.e., course code, department, and course title) and aggregated section review and rating data, which is supported by an averaging utility function. The latter task requires calling on our scraper class, and in future iterations, our model ideally avoids this to preserve modularity. Our model parses review metrics, average gpa, and class size for each section, producing a set of nearly-complete processed course data objects that we serve into our database driver. We note that *theCourseForum* dynamically stores reading, writing, and groupwork data separately from the reviews themselves. Therefore, the information is inaccessible to our web scraper in its current form, which we discuss in the Future Work section.

The outermost layer of our data collection system contains our database. We use SQLite as our driver for its ease of use and lightweight nature, enabling us to easily make modifications to our schemas and algorithms as we undergo initial building and testing on our simulation. As previously mentioned, inaccessibility to reading, writing, and group working average hours as shown on theCourseForum means that we must hard-code the values. We include a hard-coded dictionary at the top of our database driver file to be used in our SQLite methods, which we created by manually extracting values from the selected courses on *theCourseForum*. In an effort to promote scalability, the previous data collection layers include the hard-coded fields in their resulting JSON files, though they are left to 0.0 values and remain unused in the current version of our database driver.

Our outputted database table contains a row for each of the course IDs, containing basic course metadata and schemas corresponding to our model as outlined in Table 1. We chose not to scale the reading, writing, and group work values in the database to allow downstream models to test and modify the weights for each attribute.

2.1 Course Data

To achieve consistency across metric reports, we chose to utilize the metrics available on our primary data source to define our course model. Given the constraints of web scraping, course metrics are generated by averaging the values across all associated sections, meaning we rely on the assumptions that there exists little variance across sections, as well as the accuracy of historical data in defining future semesters of the course. Additionally, we manually extracted reading, writing, and groupwork time metrics and weighted the raw data to create scores that map to student preferences. We determined the weights based on the skew of the data and the results of simulation testing.

Table 2. Student Model

Unique Computing ID	3 letters, 1 number, 2 letters (xxx1xx)
GPA	4.0 Scale
Class Year	1, 2, 3, 4
Major	Engineering Department mnemonic
Course Size Preference	0-5, smaller to larger
Reading Amount Preference	0-5
Writing Amount Preference	0-5
Groupwork Amount Preference	0-5
Time Commitment	0-5 hours outside of class

2.2 Student Data

To effectively match students with courses, we decided to model certain characteristics that concretely aligned with course properties. Given the scope of the experiment, we were unable to obtain real student information, so we simulated random students with the properties described in Table 2. We based the values and distribution based on assumptions about the UVA Engineering student body, including average GPA, major distribution, etc.

3 Derivation of Preference Orderings

Once the data is collected, we must design a model mapping the data to a preference ordering for both students and courses.

The set of all students is represented as $S = \{s_1, s_2, \dots, s_n\}$.

The set of all courses is represented as $C = \{c_1, c_2, \dots, c_m\}$.

We establish the symbols depicting each variable for student i and course j :

Table 3. Variable definitions for student and course scoring functions

Symbol	Meaning
g_i	Student GPA
$\bar{g}_j$	Course average GPA
σ_i	Student size preference (0-5)
$\hat{q}_j = \min(5, \text{round}(q_j/8))$	Course size score derived from quota q_j
r_i, r_j	Student / course reading score (0-5)
w_i, w_j	Student / course writing score (0-5)
k_i, k_j	Student / course group work score (0-5)
t_i	Student time commitment (0-5 hrs)
h_j	Course hours needed (0-5 hrs)
y_i	Student class year (1-4)
$1[\cdot]$	Indicator function

3.1 Student Preferences over Courses

We define the score s_{ij} , or the score of student i for course j as:

$$s_{ij} = -8|g_i - \bar{g}_j| + 10 \cdot 1[\text{major}_i = \text{dept}_j] - |\sigma_i - \hat{q}_j| - |r_i - r_j| - |w_i - w_j| - |k_i - k_j| - 0.5 \cdot \max(0, h_j - t_i) \quad (1)$$

3.1.1 Reasoning.

- $-8|g_i - \bar{g}_j|$ – prefer courses where the student’s GPA is close to the course average
- $+10 \cdot 1[\text{major}_i = \text{dept}_j]$ – bonus for courses within the student’s major
- $-|\sigma_i - \hat{q}_j|$ – prefer courses whose size matches the student’s size preference
- $-|r_i - r_j|$ – prefer courses whose reading demand matches the student’s reading preference
- $-|w_i - w_j|$ – prefer courses whose writing demand matches the student’s writing preference
- $-|k_i - k_j|$ – prefer courses whose group work demand matches the student’s group work preference
- $-0.5 \cdot \max(0, h_j - t_i)$ – penalize courses that demand more hours than the student is willing to commit with no penalty if the student is willing to put in more time than required

3.1.2 Preference Ordering. To establish the preference ordering $\succ_i$ for student i over courses, we rank all scores s_{ij} in descending order, such that $c_j \succ_i c_k$ if and only if $s_{ij} > s_{ik}$. In our code, this is represented as a number 0 through m , where a lower number corresponds with a more preferred course.

3.2 Course Preferences over Students

$$c_{ij} = 10 \cdot 1[\text{major}_i = \text{dept}_j] + 4y_i - 3|g_i - \bar{g}_j| + 2t_i - |r_i - r_j| - |w_i - w_j| - |k_i - k_j| \quad (2)$$

3.2.1 Reasoning.

- $+10 \cdot 1[\text{major}_i = \text{dept}_j]$ – prefer students whose major matches the course department
- $+4y_i$ – prefer older students, who have less time to take courses before graduating
- $-3|g_i - \bar{g}_j|$ – prefer students with matching GPAs
- $+2t_i$ – prefer students willing to commit more time outside of class
- $-|r_i - r_j|$ – prefer students whose reading engagement matches the course’s reading demands
- $-|w_i - w_j|$ – prefer students whose writing engagement matches the course’s writing demands
- $-|k_i - k_j|$ – prefer students whose group work engagement matches the course’s group work demands

3.2.2 Preference Ordering. To establish the preference ordering $\succ_j$ for course j over students, we rank all scores c_{ij} in descending order, such that $s_i \succ_j s_k$ if and only if $c_{ij} > c_{kj}$. In our code, this is represented as a number 0 through n , where a lower number corresponds with a more preferred student.

4 Simulator

4.1 Random Serial Dictator

We first created a simulation for the random serial dictator methodology, which is most similar to how course enrollment currently occurs at UVA. In this method, described in Algorithm 1, a random order π of students is specified. In this random order, students choose the best course according to their preference ordering, provided there is capacity.

4.2 Deferred Acceptance

In deferred acceptance, students repeated propose to courses that tentatively accept them or reject them until the matching becomes stable. In each round each unmatched student proposes to their most preferred course that they have not been rejected from, and courses only reject in favor of more preferred students. This continues until all students are matched or courses reach capacity. This algorithm is described in Algorithm 2. We adapted code from [Morita, 2022] in our algorithm.

ALGORITHM 1: Random Serial Dictatorship (RSD)**Input:** Student preference matrix P^S , course quotas $q_j \forall c_j \in C$ **Output:** Matching $\mu : S \rightarrow C$ Draw a uniformly random ordering π over S ; $\mu \leftarrow \emptyset$; $\text{remaining}_j \leftarrow q_j \quad \forall c_j \in C$;**for each student $s_{\pi(i)}$ in order do** **for each course c_j in $s_{\pi(i)}$'s preference ordering do** **if $\text{remaining}_j > 0$ then** $\mu(s_{\pi(i)}) \leftarrow c_j$; $\text{remaining}_j \leftarrow \text{remaining}_j - 1$;

break;

end **end****end****return μ ;****ALGORITHM 2:** Deferred Acceptance (DA)**Input:** Student preference matrix P^S , course preference matrix P^C , course quotas $q_j \forall c_j \in C$ **Output:** Matching $\mu : S \rightarrow C$ $\mu \leftarrow \emptyset$; $\text{available}_i \leftarrow C \quad \forall s_i \in S$; $\text{unmatched} \leftarrow \emptyset$;**repeat** **for each student $s_i \notin \text{unmatched}$ do** **if $\text{available}_i = \emptyset$ then** $\text{unmatched} \leftarrow \text{unmatched} \cup \{s_i\}$; **else** $c^* \leftarrow \arg \min_{c_j \in \text{available}_i} P_{ij}^S$; $\mu(s_i) \leftarrow c^*$; **end** **end** **for each oversubscribed course c_j where $|\mu^{-1}(c_j)| > q_j$ do** Sort $\mu^{-1}(c_j)$ by P_{ij}^C descending; **for each rejected student s_i beyond quota q_j do** $\mu \leftarrow \mu \setminus \{(s_i, c_j)\}$; $\text{available}_i \leftarrow \text{available}_i \setminus \{c_j\}$; **end** **end** $\text{unmatched} \leftarrow \{s_i : \mu(s_i) \text{ is defined}\}$;**until $|\text{unmatched}| = |S|$;****return μ ;****5 Performance Evaluation**

After performing both RSD and DA with our derived preference orderings, we obtained the following results:

Table 4. Results of RSD and DA Simulation

Metric	RSD	DA
Blocking pairs	1337	0
Pareto-improving swaps	0	0
Mean preference rank	0.481	0.486
Top choice %	64.2%	64.7%
Unmatched students	0	0

This shows how DA might be a better algorithm in this case, because while metrics like the top-choice percentage and mean preference rank are similar, we eliminate 1337 blocking pairs in the simulation. This demonstrates a meaningful improvement for DA over RSD in this sample. This would have to be generalized to more data in order to be conclusive, but this shows a promising pattern for DA.

6 Conclusions

In this paper, we develop a student-proposing course enrollment system modeled after a many-to-one matching-market. UVA student evaluations of courses are collected to characterize course preferences, student preference schemas are defined to create a direct mapping between the two model characteristics. Data is served into a preference ordering algorithm to normalize metrics, and the outputted rankings are used to simulate deferred acceptance and random serial dictatorship matching markets. A matching mechanism analysis class initiates and evaluates the results of the simulation under DA and RSD conditions.

7 Future Work

Given the scope of this project and course, we identified possible areas of further refinement and research for the future:

- (1) **Integrate UVA Student Data:** Using data about real students was difficult for this project, given the timeline and concerns about potential data privacy, since we would require large amounts to make strong conclusions. If possible, it would help to gather real student data and their own course preferences to have a more accurate depiction of the UVA process and possibly compare our system's predicted preferences to their self-reported ones. With real data, this would likely lead to different insights leading us to adjust the weights for each component of the score.
- (2) **Expand Characteristics Modeled:** Due to computational and data availability constraints, our project was limited in the number and type of characteristics we could consider. In the future, it would be interesting to incorporate more features, as well as other types, such as sentiment detected from doing Natural Language Processing on course reviews.

References

2026. *The Course Forum*. <https://thecourseforum.com/>. Accessed: 2026.
- Kyosuke Morita. 2022. Deferred Acceptance School Choice. https://github.com/kyosek/deferred_acceptance_school_choice/tree/master. Copyright (c) 2022 Kyosuke Morita. Licensed under the MIT License.

A Code Base Information

Our code base can be found at https://github.com/louisarawles/enrollment_matching_market/.

It is published under the MIT License.